Analysis of Algorithm (CSN13400)
ASSIGNMENT-6

Name : Aayansh Niranjan

Reg_No.: 20253004

Date :27/09/2026

Q1. You are working for a retail company, and you have been provided with sales data for various products. Each product has a unique identifier (ID) which ranges from 0 to 1000. Your task is to analyse the sales data to understand which products are the most popular based on the frequency of sales. By using an efficient algorithm, you need to sort the product IDs based on the frequency of sales such that:

- Products with higher sales frequency come first.
- If two products have the same sales frequency, the product with the smaller ID comes first.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 1000

int main(){
   int freq[N+1]={0};
   int count[N+1]={0};
   int *bucket[N+1]={NULL};

   srand(time(NULL));

   for(int i=0;i<=N;i++)
      freq[i]=rand()%100+1;

   for(int i=0;i<=N;i++)
      if(freq[i]>0)
         count[freq[i]]++;

   for(int i=1;i<=N;i++)
      if(count[i]>0)
         bucket[i]=malloc(count[i]*sizeof(int));

   int index[N+1]={0};

   for(int i=0;i<=N;i++){
      if(freq[i]>0){
         int f=freq[i];
         bucket[f][index[f]++]=i;
      }
   }

   printf("Product ID\tFrequency\n");

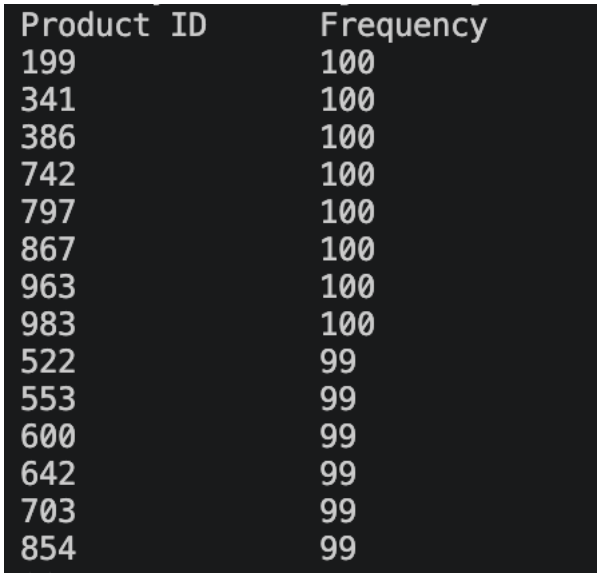
   for(int f=N;f>=1;f--){
      if(count[f]>0){
         for(int j=0;j<count[f];j++){
            int id=bucket[f][j];
            printf("%d\t\t%d\n",id,f);
         }
      }
   }

   for(int i=1;i<=N;i++)
      free(bucket[i]);

   return 0;
}
```

A1. CODE:

OUTPUT:



Product ID	Frequency
199	100
341	100
386	100
742	100
797	100
867	100
963	100
983	100
522	99
553	99
600	99
642	99
703	99
854	99

Q2. You are given an array of 10000 integers where each integer is represented as a 32-bit unsigned binary number. You need to sort the array based on the binary representation of the numbers in lexicographical order. Use a suitable sorting algorithm to implement this efficiently. (All integers fit within 32-bit unsigned integers). Calculate the required time in milliseconds and compare with any two existing sorting based-approaches.

A2. CODE:

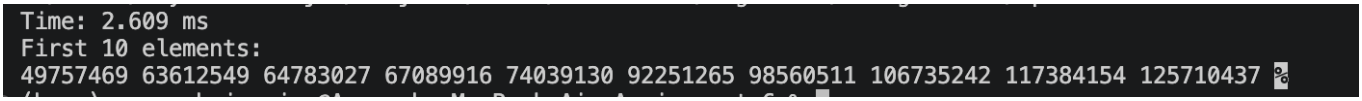
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 10000

void radixSort(unsigned int a[], int n) {
    unsigned int *b=malloc(n*sizeof(unsigned int));
    for(int bit=0;bit<32;bit++){
        int count[2]={0,0};
        for(int i=0;i<n;i++){count[(a[i]>>bit)&1]++;}
        int pos[2]={0,count[0]};
        for(int i=0;i<n;i++){
            int x=(a[i]>>bit)&1;
            b[pos[x]++]=a[i];
        }
        for(int i=0;i<n;i++){
          a[i]=b[i];
        }
    }
    free(b);
}

int main() {
    unsigned int a[N];
    srand(time(NULL));
    for(int i=0;i<N;i++){
        a[i]=((unsigned int)rand()<<16) | rand();
    }
    clock_t start = clock();
    radixSort(a, N);
    clock_t end = clock();
    printf("Time: %.3f ms\n",(double)(end-start)*1000/CLOCKS_PER_SEC);
    printf("First 10 elements:\n");
    for(int i=0;i<10;i++)
        printf("%u ",a[i]);
    return 0;
}
```

OUTPUT:


Time: 2.609 ms
First 10 elements:
49757469 63612549 64783027 67089916 74039130 92251265 98560511 106735242 117384154 125710437

Q3. You are tasked with analyzing the performance of a network by sorting and categorizing latency measurements. You have an array of n latency measurements in milliseconds, each ranging from 0 to 1000 milliseconds. Your goal is to use a fast-sorting algorithm to efficiently categorize these measurements into specific time ranges and then provide the following information:

- Categorized Latency Ranges: Divide the latencies into predefined sub-intervals such as: [0, 100), [100,200), [200,300), and so on up to [900, 1000]. Each sub-interval represents a time range.
- Count of Measurements in Each Sub-interval: Determine how many measurements fall into each of these time ranges.
- Sorted List of Measurements in Each Sub-interval: For each sub-interval, provide a sorted list of latencies that fall into that range.
- Median Latency for Each Sub-interval: Calculate the median latency for each subinterval. The median is the middle value when the latencies in that subinterval are sorted.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
   int n=20;
   int a[20];
   int c[1001]={0};
   srand(time(0));

   for(int i=0;i<n;i++){
      a[i]=rand()%1001;
      c[a[i]]++;
   }

   printf("Random data: ");
   for(int i=0;i<n;i++)
      printf("%d ",a[i]);

   for(int r=0;r<10;r++){
      int l=r*100;
      int u=(r==9) ? 1000:l+99;
      int b[20];
      int k=0;

      for(int x=l;x<=u;x++)
         while(c[x]--){
           b[k++]=x;
         }
      printf("\n\n[%d,%d] Count=%d\nSorted: ",l,u,k);
      for(int i=0;i<k;i++){printf("%d ",b[i]);}
      if(k==0)
         printf("\nMedian: none");
      else if(k%2)
         printf("\nMedian: %d",b[k/2]);
      else
         printf("\nMedian: %.2f",(b[k/2-1]+b[k/2])/2.0);
   }
   printf("\n");
   return 0;
}
```

A3. CODE:

```
Random data: 386 103 18 97 935 452 918 180 626 861 799 289 834 453 888 771 440 389 909 253

[0,99] Count=2
Sorted: 18 97
Median: 57.50

[100,199] Count=2
Sorted: 103 180
Median: 141.50

[200,299] Count=2
Sorted: 253 289
Median: 271.00

[300,399] Count=2
Sorted: 386 389
Median: 387.50

[400,499] Count=3
Sorted: 440 452 453
Median: 452

[500,599] Count=0
Sorted:
Median: none

[600,699] Count=1
Sorted: 626
Median: 626

[700,799] Count=2
Sorted: 771 799
Median: 785.00

[800,899] Count=3
Sorted: 834 861 888
Median: 861

[900,1000] Count=3
Sorted: 909 918 935
Median: 918
```

OUTPUT: